

Question 1

A data engineer is asked to investigate why a Lakeflow Job experienced a gradual increase in execution duration over several production runs. The engineer needs a feature that visualizes execution trends and compares metrics from prior job executions. Which option should be selected?

A. Dashboards

B. Run History

C. Catalog Explorer

D. Query History

Correct answer: B

B. Run History

Explanation

The **Run History** (found inside the Lakeflow Jobs UI under the **Runs** tab) is specifically designed to track and monitor the performance of workflows over time.

- **Visualizing Execution Trends:** It provides a matrix and list view of recent, active, and completed job runs, along with duration graphs that allow a data engineer to easily spot patterns like a **gradual increase in execution duration** (performance regression).

- **Comparing Metrics:** It allows you to select and compare past runs side-by-side to look at performance indicators (e.g., rows read/written, query counts, task-level durations) to determine if the slowdown is caused by data volume growth, data skew, or changing workloads.

Why the Other Options are Incorrect

- **Query History:** While Query History tracks individual SQL queries executed across SQL warehouses (showing query profiles, execution times, and performance bottlenecks), it is intended for ad-hoc or warehouse-level query debugging. It does not provide the unified, automated orchestration metrics or sequential multi-task trend monitoring inherent to **Lakeflow Jobs**.

- **Catalog Explorer:** This tool is used strictly for data governance, discovering data assets, managing permissions (via Unity Catalog), and examining schema definitions or data lineage. It does not track or visualize job execution metrics or runtime durations.

- **Dashboards:** Lakeflow Dashboards are used to build business intelligence (BI) data visualizations and reports for end-users. While you could technically build a custom dashboard by querying underlying

system tables (like `system.lakeflow.job_run_timeline`), it is not an out-of-the-box feature built natively into the Job interface for checking immediate execution trends.

Question 2

A data engineering team is working on a user activity events table stored in Unity Catalog. Queries often involve filters on multiple columns like **user_id** and **event_date**.

Which data layout technique should the team implement to avoid expensive table scans?

- A. Use Z-order indexing on the user_id
- B. Use partitioning on the user_id column, along with Z-order indexing on the event_date column.
- C. Use partitioning on the event_date column.
- D. Use liquid clustering on the combination of user_id and event_date

Correct answer: D

D. Use liquid clustering on the combination of user_id and event_date

Explanation

In this scenario, using liquid clustering on the combination of user_id and event_date is the best choice to avoid expensive scans. This technique incrementally optimizes data layout based on both columns, efficiently supporting filters on these columns and avoiding costly table scans.

Partitioning only on **event_date** helps queries filtering by date but doesn't optimize filtering by user_id, leading to potential full scans within partitions. Z-order indexing on **user_id** optimizes queries filtering on **user_id** but ignores event_date filtering, resulting in inefficient scans when filtering by date. Lastly, partitioning on user_id + Z-order on event_date supports filtering on both columns but can create many small partitions (if users are numerous), causing management and performance issues.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

Question 3

In Lakeflow Jobs, which compute option helps reduce operational overhead and idle cluster costs?

- A. Classic all-purpose compute
- B. SQL warehouses
- C. Serverless jobs compute
- D. Classic jobs compute

Correct answer: C

C. Serverless jobs compute

Explanation

Serverless jobs compute With serverless compute, Databricks fully manages the infrastructure in its own compute plane (handling scaling, patching, and resource allocation). It dynamically provisions exactly what the job needs and terminates immediately when the job finishes. It dramatically reduces **operational overhead** because you no longer need to configure cluster sizing or VM types, and it virtually eliminates **idle cluster costs** because you are only billed for the exact duration your workload runs (with minimal startup delay).

Why the other options are incorrect:

- **SQL warehouses:** While serverless SQL warehouses exist, they are optimized specifically for SQL analytics, BI tooling, and running ad-hoc queries, rather than natively orchestrating multi-step data pipelines within Lakeflow Jobs.

- **Classic all-purpose compute:** This option is meant for interactive, ad-hoc development (like running code cells inside a notebook). Running production jobs on all-purpose compute incurs significantly higher Databricks Unit (DBU) costs and leaves you highly vulnerable to idle costs if the cluster doesn't shut down promptly after the job finishes.

- **Classic jobs compute:** Though cheaper than all-purpose compute, classic jobs require you to manually size, configure, and manage instance types or pools. They also suffer from a 4-to-6 minute startup latency on your cloud provider's plane, during which you are still paying for infrastructure initialization.

Question 4

A data engineer is using the following Auto Loader stream to incrementally ingest JSON files from an object cloud storage into a Delta table:

```
spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "json")
  _____
  .load(cloud_storage_path)
.writeStream
  .option("checkpointLocation", checkpoint_path)
  .toTable(target_table)
```

Files may occasionally be re-uploaded under the same path and filename to apply data corrections. Which option correctly fills in the blank in order to load those corrections into the table?

- A. `.option("mergeSchema", "true")`
- B. `.option("cloudFiles.includeExistingFiles", "true")`
- C. `.option("cloudFiles.force", "true")`
- D. `.option("cloudFiles.allowOverwrites", "true")`

Correct answer: D

D. `.option("cloudFiles.allowOverwrites", "true")`

Explanation

By default, Databricks Auto Loader uses a standard `cloudFiles.allowOverwrites = false` configuration. Under this default behavior, each file path is strictly processed **exactly once**. If a file is updated, appended to, or re-uploaded under the exact same path and filename, Auto Loader will normally ignore it.

Setting `cloudFiles.allowOverwrites` to "true" instructs Auto Loader to look at both the file path and the file's last-modified timestamp. When a file is overwritten or re-uploaded with data corrections, Auto Loader detects the updated timestamp and re-ingests the entire file into the stream.

Question 5

A data engineer is validating incoming transaction data in a Databricks SQL pipeline. Some records contain unexpected symbols in numeric fields. The engineer runs the following query:

```
SELECT COALESCE(TRY_CAST("100$" AS INT), 0)
```

Which of the following describes the result of this query?

- A. NULL
- B. 0
- C. An error is thrown
- D. 100

Correct answer: B

B. 0

Explanation

COALESCE is a built-in SQL function that evaluates a list of arguments in order and **returns the very first non-NULL value** it finds, in this case, 0.

Here is exactly how the Databricks SQL pipeline processes this query from the inside out:

- TRY_CAST("100\$" AS INT): The TRY_CAST function attempts to cast the string "100\$" to an integer. Because the string contains an unexpected non-numeric symbol (\$), a standard CAST would normally throw a runtime error. However, TRY_CAST is specifically designed to handle formatting mismatches gracefully; when it encounters a value it cannot cast, it returns **NULL** instead of failing the pipeline.

- COALESCE(..., 0): The COALESCE function evaluates the arguments in order and returns the first non-null value. Since the first argument (TRY_CAST outcome) is NULL, COALESCE falls back to the second argument, which is 0.

Reference

Question 6

A data engineer has deployed a bundle with a job resource keyed as `sales_job` and wants to execute it in the target workspace.

Which Databricks CLI command can the data engineer use to achieve this goal?

A. `databricks bundle deploy --run sales_job`

B. `databricks bundle trigger sales_job`

C. `databricks bundle run sales_job`

D. `databricks bundle execute sales_job`

Correct answer: C

C. `databricks bundle run sales_job`

Explanation

When working with Declarative Automation Bundles (DABs), you first use `databricks bundle deploy` to provision the resources into your designated workspace. Once deployed, the **run** command is used to trigger a specific job or pipeline resource defined within your configuration files.

Syntax Breakdown

```
databricks bundle run <resource-key> [flags]
```

- **sales_job**: This corresponds to the resource key assigned to the job inside your `databricks.yml` (or included resources) configuration file.

- If you have multiple targets configured (e.g., `dev`, `prod`) and want to target a non-default workspace, you can append the target flag: `databricks bundle run sales_job -t prod`.

Why other options are incorrect:

- `databricks bundle execute`: The `execute` subcommand does not exist under the `bundle` command group.

- `databricks bundle trigger`: While it sounds intuitive for triggering a job, it is not a valid Databricks CLI `bundle` command.

- `databricks bundle deploy --run`: The `deploy` command is strictly meant for validating and copying your project bundle assets into the target environment; it does not feature an inline `--run` flag to simultaneously execute jobs.

Question 7

In the Medallion Architecture, which of the following statements best describes the Gold layer tables?

- A. They maintain raw data ingested from various sources
- B. The table structure in this layer resembles that of the source system table structure with any additional metadata columns like the load time, and input file name.
- C. They represent a filtered, cleaned, and enriched version of data
- D. They provide business-level aggregations that power analytics, machine learning, and production applications

Correct answer: D

D. They provide business-level aggregations that power analytics, machine learning, and production applications

Explanation

Gold layer is the final layer in the multi-hop architecture, where tables provide business level aggregates often used for reporting and dashboarding, or even for Machine learning.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

- [Hands-on](#)

Question 8

Which of the following techniques allows Auto Loader to track the ingestion progress and store metadata of the discovered files ?

A. Watermarking

B. Photon engine

C. COPY INTO

D. Checkpointing

Correct answer: D

D. Checkpointing

Explanation

Auto Loader keeps track of discovered files using checkpointing in the checkpoint location. Checkpointing allows Auto loader to provide exactly-once ingestion guarantees.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

- [Hands-on](#)

Question 9

“A mechanism to dynamically apply row filters and column masks using governed tags. It offers a single catalog-level definition that automatically applies to all current and future tables without per-table configuration.”

Which of the following Unity Catalog features is being described in the above statement?

A. Attribute-Based Access Control (ABAC) policies

B. Liquid Clustering

C. Dynamic views

D. Predictive Optimization

Correct answer: A

A. Attribute-Based Access Control (ABAC) policies

Explanation

In Databricks Unity Catalog, ABAC extends the governance framework beyond standard user or role boundaries. It allows data administrators to use **governed tags** (attributes attached to schemas, tables, or columns) to dynamically apply Row Filters and Column Masks. Instead of manually applying rules to every individual asset, a single policy defined at the catalog level evaluates these tags and automatically secures all matching columns or rows-both for existing data assets and any future tables created with those tags.

Why the other options are incorrect:

- **Dynamic views:** While dynamic views can be configured with conditional logic (like `is_account_group_member()`) to mask columns or filter rows, they require creating and maintaining individual view definitions on a per-case basis. They do not automatically scale catalog-wide across future tables via a tag-based declaration.

- **Predictive Optimization:** This feature automatically manages background maintenance tasks for Delta tables (such as `OPTIMIZE` and `VACUUM` commands) to ensure peak query performance. It is not an access control or data masking mechanism.

- **Liquid Clustering:** This is a next-generation data layout and indexing technique designed to replace traditional Hive partitioning and Z-ordering. It accelerates query performance by dynamically clustering data based on specified columns, but it plays no role in data governance or security policies.

Question 10

A junior data engineer is exploring a large dataset in Databricks and executes `df.describe()` to better understand the data.

A senior data engineer suggested using `df.summary()` instead.

Which advantage does `df.summary()` provide over `df.describe()` for data exploration?

- A. `df.summary()` generates visual charts and interactive plots for each column in the dataset.
- B. `df.summary()` automatically cleans null values and removes outliers before computing statistics.
- C. `df.summary()` offers an AI-generated summary of the dataset that provides insights, patterns, and recommendations based on the underlying data values.
- D. `df.summary()` provides extended statistics such as approximate quartiles (25%, 50%, 75%), which are not available in `df.describe()`.

Correct answer: D

D. `df.summary()` provides extended statistics such as approximate quartiles (25%, 50%, 75%), which are not available in `df.describe()`.

Explanation

Both `df.describe()` and `df.summary()` are used for quick exploratory data analysis, but they calculate different sets of statistics by default.

- `df.describe()`: Computes a basic, fixed set of statistics for numeric and string columns: count, mean, stddev, min, and max.

- `df.summary()`: Computes everything `describe()` does, but **automatically includes approximate quartiles (25%, 50%, and 75%)**.

Furthermore, `df.summary()` is more flexible because it allows you to pass specific statistics as arguments if you only want a subset of the data (e.g., `df.summary("count", "50%", "95%")`), whereas `describe()` accepts a list of column names but a fixed set of metrics.

Question 11

A data engineer needs to execute a scheduled Python notebook on an hourly basis. This job involves processing relatively small volumes of data. The execution environment should be efficient, require no cluster pre-warming, and deliver consistent, reliable performance.

Considering these requirements, which compute option is recommended for executing this job?

A. Classic All-purpose Cluster

B. Serverless Job Compute

C. Serverless SQL Warehouse

D. Classic Job Cluster

Correct answer: B

B. Serverless Job Compute

Explanation

Serverless Job Compute offers an ideal balance of efficiency, automation, and performance consistency for scheduled jobs. It eliminates the need for cluster pre-warming by dynamically provisioning resources, thereby reducing costs and operational overhead. It is designed specifically to support batch jobs, including Python scripts, making it the most suitable and modern choice for this scenario.

Databricks recommends using serverless compute for all job tasks. You can also specify different compute types for tasks in a job, which might be required if a task type is not supported by serverless compute for workflows.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

Question 12

A data engineer wants to develop a standard connector using Lakeflow Connect to ingest sales transactions into a Unity Catalog table nightly. The engineer needs to retain historical data in the table for 30 days, while preventing duplicates across repeated loads.

Which command can the data engineer use to meet these requirements?

A. INSERT INTO

B. COPY INTO

C. MERGE INTO

D. INSERT OVERWRITE

Correct answer: C

C. MERGE INTO

Explanation

To meet the requirements of retaining 30 days of historical data while preventing duplicates during nightly loads, the data engineer should use the `MERGE INTO` command.

The `MERGE INTO` statement (often referred to as an "upsert") allows you to join a source data set with a target table and perform multiple actions based on whether a match is found.

- **Deduplication:** You can define a unique key (e.g., `transaction_id`). If the ID already exists in the target table, you can choose to update the record or do nothing; if it doesn't exist, you insert it.

- **Retention:** Since `MERGE` only modifies or adds specific rows, the rest of your historical data (up to your 30-day limit) remains untouched.

Comparison of Options

Command	Behavior	Why it fails these requirements
CREATE OR REPLACE	Drops the existing table and creates a new one.	You would lose all 30 days of historical data every night.
INSERT OVERWRITE	Replaces data in the table (or specific partitions).	Without complex partitioning, it typically wipes the entire table, losing history.
COPY INTO	Loads files from a folder into a table.	While it tracks which files have been processed, it does not handle row-level de-duplication if the same record appears in different files.
MERGE INTO	Conditionally updates, deletes, or inserts rows.	Matches both needs: Prevents duplicates via keys and preserves history.

Question 13

A healthcare analytics company receives medical imaging metadata files every few seconds into cloud object storage. Due to compliance requirements, the storage location has accumulated thousands of archived JSON files over several months. The engineering team wants a streaming ingestion solution that can:

- Process all archived files exactly once
- Automatically detect new incoming files in near real time
- Minimize storage API calls and metadata listing overhead

Which ingestion strategy is most appropriate?

- A.** Use COPY INTO initially to efficiently process the existing files, then use Auto Loader with file notification mode to process new files.
- B.** Use Auto Loader with file notification mode to enable event-based file detection for current and future files.
- C.** Use a CTAS statement initially to load the existing files in batch mode, then use Auto Loader to incrementally process new files.
- D.** Use Auto Loader in directory listing mode because archived files must always be discovered through full directory scans before streaming can begin.

Correct answer: B

B. Use Auto Loader with file notification mode to enable event-based file detection for current and future files.

Explanation

Databricks **Auto Loader** (using the cloudFiles format) is explicitly designed to handle exactly this scenario: seamlessly transitioning from loading thousands/millions of existing historical files to streaming newly arrived files in near real-time, all while providing **exactly-once processing guarantees**.

Here is why this strategy fulfills all the requirements:

- **Process Archived Files Exactly Once:** When you start an Auto Loader stream with file notification mode, Auto Loader automatically performs an initial backfill by listing the directory to discover all existing (archived) files. It records their metadata in a scalable internal key-value store (checkpoint), ensuring every file is processed **exactly once**, even if the stream fails and restarts.
- **Near Real-Time Auto-Detection:** By choosing **file notification mode**, Auto Loader sets up a cloud

notification service (such as AWS SNS/SQS, Azure Event Grid, or Google Cloud Pub/Sub) that automatically listens to cloud storage file events. As soon as a new JSON metadata file is uploaded, the stream processes it immediately.

- **Minimize Storage API Calls & Overhead:** This is the core advantage of file notification mode over directory listing mode. As a storage bucket grows to contain thousands or millions of files, performing full directory scans (LIST operations) becomes incredibly slow and expensive. File notification mode allows Auto Loader to discover new files completely by listening to event queues, drastically minimizing metadata listing overhead and cloud storage API costs.

Why the other options are incorrect:

- **Directory listing mode:** While it works, it requires continuous, resource-heavy listing scans over the directory to detect changes, which violates the requirement to minimize storage API calls and metadata listing overhead.

- **COPY INTO initially, then Auto Loader:** This introduces unnecessary operational complexity. Auto Loader handles both existing files and incoming files automatically in a single unified architecture without needing a secondary tool.

- **CTAS statement initially, then Auto Loader:** Similar to COPY INTO, using a manual batch step like Create Table As Select (CTAS) creates overhead. It forces you to manually manage the cut-off point between the historical batch and the real-time stream, risking missing files or duplicating data.

Question 14

A data engineering team is ingesting JSON logs from cloud storage into a Delta table using the following code:

```
COPY INTO target_table
FROM 'abfss://logs@staging.dfs.core.windows.net/'
FILEFORMAT = JSON
_____;
```

The source contains both old and newly updated files and the team wants to filter them by their last modification time to load only recent files.

Which option correctly fills in the blank to meet the specified requirement?

- A. `FORMAT_OPTIONS ('modificationTime' > 'ts')`
- B. `FORMAT_OPTIONS (_metadata.file_modification_time > 'ts')`
- C. `FORMAT_OPTIONS ('modifiedAfter' = 'ts')`
- D. `FORMAT_OPTIONS (file.modification_time > 'ts')`

Correct answer: C

C. `FORMAT_OPTIONS ('modifiedAfter' = 'ts')`

Explanation

In Databricks SQL, the `COPY INTO` command allows you to specify generic file data source reader options inside the `FORMAT_OPTIONS` clause.

To filter and ingest only files that have been modified after a specific date and time, Databricks provides the generic format option `modifiedAfter`. It accepts a timestamp string (e.g., '2026-01-01 00:00:00.000000 UTC+0') to filter out files that are older than the specified cutoff.

The required syntax uses key-value assignments inside `FORMAT_OPTIONS`:

```
COPY INTO target_table
FROM 'abfss://logs@staging.dfs.core.windows.net/'
FILEFORMAT = JSON
FORMAT_OPTIONS ('modifiedAfter' = 'ts');
```

The other options are incorrect because they represent invalid syntax formats

Question 15

Which of the following SQL commands will append this new row to the existing Delta table **users**?

user_id	name	age
0015	Adam	23

- A. INSERT VALUES ("0015", "Adam", 23) INTO users
- B. INSERT INTO users VALUES ("0015", "Adam", 23)
- C. APPEND VALUES ("0015", "Adam", 23) INTO users
- D. APPEND INTO users VALUES ("0015", "Adam", 23)

Correct answer: B

B. INSERT INTO users VALUES ("0015", "Adam", 23)

Explanation

INSERT INTO allows inserting new rows into a Delta table. You specify the inserted rows by value expressions or the result of a query.

Study materials from our exam preparation course on Udemy:

- [Hands-on](#)

Question 16

A data engineer is optimizing a Spark job in Databricks that joins a large fact table with a small dimension table. They want to ensure that Spark automatically sends small tables up to 100 MB to all executors to improve join performance. What configuration should they set?

- A. `spark.conf.set("spark.sql.shuffle.partitions", 100)`
- B. `spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 104857600)`
- C. `spark.conf.set("spark.sql.broadcastTimeout", 100)`
- D. `spark.conf.set("spark.executor.memory", 104857600)`

Correct answer: B

- B. `spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 104857600)`

Explanation

This `spark.sql.autoBroadcastJoinThreshold` property configures the maximum size, in bytes, for a table that Spark will automatically broadcast to all worker nodes when performing a join. By broadcasting a small dimension table, Spark avoids an expensive shuffle operation of the large fact table, significantly boosting performance. This is known as a **Broadcast Hash Join**.

- **The Math:** The configuration requires the value to be set in bytes.

$100 \text{ MB} = 100 * 1024 * 1024 \text{ bytes} = 104,857,600 \text{ bytes}$

Why the other options are incorrect

- `spark.sql.broadcastTimeout`: This sets the timeout change (in seconds) for how long an executor will wait for the broadcast dataset before throwing an error. It doesn't control the size threshold.

- `spark.sql.shuffle.partitions`: This controls the default number of partitions used when shuffling data for joins or aggregations (the default is typically 200).

- `spark.executor.memory`: This controls the total amount of memory allocated to each executor process, not a specific join behavior.

Question 17

A data engineer has a custom-location schema named **db_hr**, and they want to know where this schema was created in the underlying storage.

Which of the following commands can the data engineer use to complete this task?

A. `SELECT location FROM db_hr.db`

B. `DESCRIBE EXTENDED db_hr`

C. `DESCRIBE db_hr`

D. `DESCRIBE DATABASE db_hr`

Correct answer: D

D. `DESCRIBE DATABASE db_hr`

Explanation

The `DESCRIBE DATABASE` or `DESCRIBE SCHEMA` returns the metadata of an existing schema (database). The metadata information includes the database's name, comment, and location on the filesystem. If the optional `EXTENDED` option is specified, database properties are also returned.

Syntax:

```
DESCRIBE DATABASE [ EXTENDED ] database_name
```

Study materials from our exam preparation course on Udemy:

[Hands-on](#)

Question 18

A data engineer has the following query in a Lakeflow Spark Declarative Pipeline (formerly Delta Live Tables):

```
CREATE STREAMING TABLE sales_silver
AS
  SELECT store_id, total + tax AS total_after_tax
  FROM sales_bronze
```

The pipeline is failing to start due to an error in this query.

Which of the following changes should be made to this query to successfully start the DLT pipeline ?

A.

```
CREATE STREAMING TABLE sales_silver
AS
  SELECT store_id, total + tax AS total_after_tax
  FROM STREAMING sales_bronze
```

B.

```
CREATE STREAMING TABLE sales_silver
AS
  SELECT store_id, total + tax AS total_after_tax
  FROM STREAM(sales_bronze)
```

C.

```
CREATE STREAM TABLE sales_silver
AS
  SELECT store_id, total + tax AS total_after_tax
  FROM STREAM(LIVE.sales_bronze)
```

D.

```
CREATE STREAMING TABLE sales_silver
AS
  SELECT store_id, total + tax AS total_after_tax
  FROM STREAMING(sales_bronze)
```

Correct answer: B

Explanation

In DLT* pipelines, You can stream data from other tables in the same pipeline by using the `STREAM()` function**. In this case, you must define a streaming table using the `CREATE STREAMING TABLE` syntax.

```
CREATE STREAMING TABLE table_name
AS SELECT *
  FROM STREAM(source_table)
```

* Please note that Delta Live Tables (DLT) has been recently renamed to Lakeflow Declarative Pipeline, however, the current exam version may still refer to it as Delta Live Tables.

** A recent syntax update now also supports the use of `STREAM` as a keyword.

```
CREATE STREAMING TABLE table_name
AS SELECT *
  FROM STREAM source_table;
```

Study materials from our exam preparation course on Udemy:

[Hands-on](#)

Question 19

Given the following table **faculties**

faculty_id	faculty_name	students
F001	Faculty of Medicine	[{"student_id": "S000002501", "total_courses": "6"}, {"student_id": "S000004478", "total_courses": "2"}, {"student_id": "S000001572", "total_courses": "5"}, {"student_id": "S000003859", "total_courses": "1"}]
F002	Faculty of Economics	[{"student_id": "S000007415", "total_courses": "3"}, {"student_id": "S000001177", "total_courses": "4"}, {"student_id": "S000005631", "total_courses": "7"}, {"student_id": "S000001003", "total_courses": "6"}]
F003	Faculty of Engineering	[{"student_id": "S000007251", "total_courses": "2"}, {"student_id": "S000002415", "total_courses": "5"}]

Fill in the following blank to get the students enrolled in less than 3 courses from the array column **students**

```
SELECT
    faculty_id,
    students,
    _____ AS few_courses_students
FROM faculties
```

- A. TRANSFORM (students, i -> i.total_courses < 3)
- B. FILTER (students, i -> i.total_courses < 3)
- C. TRANSFORM (students, total_courses < 3)
- D. FILTER (students, total_courses < 3)

Correct answer: B

B. FILTER (students, i -> i.total_courses < 3)

Explanation

`filter(input_array, lambda_function)` is a higher order function that returns an output array from an input array by extracting elements for which the predicate of a lambda function holds.

Example:

Extracting odd numbers from an input array of integers:

```
SELECT filter(array(1, 2, 3, 4), i -> i % 2 == 1);
```

output: [1, 3]

Study materials from our exam preparation course on Udemy:

[Hands-on](#)

Question 20

Which of the following compute resources is available in Databricks SQL?

- A. SQL engines
- B. SQL warehouses
- C. Single-node clusters
- D. Multi-nodes clusters

Correct answer: B

B. SQL warehouses

Explanation

Compute resources are infrastructure resources that provide processing capabilities in the cloud. A SQL warehouse is a compute resource that lets you run SQL commands on data objects within Databricks SQL.

Study materials from our exam preparation course on Udemy:

[Hands-on](#)

Question 21

Which of the following services provides a data warehousing solution in the Databricks Intelligence Platform?

A. Lakeflow Connect

B. SQL warehouse

C. Unity Catalog

D. Databricks SQL

Correct answer: D

D. Databricks SQL

Explanation

Databricks SQL (DB SQL) is a data warehouse on the Databricks Lakehouse Platform that lets you run all your SQL and BI applications at scale.

Study materials from our exam preparation course on Udemy:

[Hands-on](#)

Question 22

A data engineer runs a Spark job that performs a wide transformation where data need to be reshuffled across partitions, but it fails with an `ExecutorLostFailure` error message.

Which TWO actions should the data engineer take to fix this issue?

- A. Increase executor memory
- B. increase the number of shuffle partitions
- C. increase the driver memory
- D. Enable the Photon engine
- E. Increase the number of cores per executor

Correct answers: A, B

- A. Increase executor memory
- B. increase the number of shuffle partitions

Explanation

An `ExecutorLostFailure` during a wide transformation (which involves a shuffle such as a join of two large tables) is almost always a result of the **YARN/Kubernetes NodeManager killing the executor container for exceeding its allocated memory limits** (often triggered by an Out Of Memory / OOM error). This indicates an executor memory exhaustion during shuffle.

Here is how the two chosen actions directly address the root causes:

- **Increase executor memory:** Shuffling requires Spark to buffer and aggregate data records in the executor memory before transferring them across the network. If the data volume within a partition is too large, the executor will run out of memory space, causing the host operating system or cluster manager to terminate the executor process. Providing more memory prevents this crash.
- **Increase the number of shuffle partitions:** By default, Spark sets `spark.sql.shuffle.partitions` to 200. If your dataset is large, dividing it into only 200 chunks results in massive individual partitions (skewed or oversized partitions). Increasing this number breaks the dataset into a higher number of **smaller, more manageable chunks**. This reduces the memory footprint required by an individual executor to process any single partition at a given time.

Why the other options are incorrect

- **Increase the driver memory:** The driver memory is only responsible for scheduling, maintaining the execution plan, and collecting final results (e.g., via `.collect()`). The actual reshuffling of data happens exclusively on the executors, so driver memory will not resolve an executor-level crash during a wide transformation.

- **Increase the number of cores per executor:** Increasing cores without increasing memory will actually worsen the problem. More cores mean more concurrent tasks running inside the same executor memory pool, reducing the amount of memory available per task and accelerating OOM failures.

- **Enable the Photon engine:** While Databricks' Photon engine optimizes query execution speed, it does not inherently solve systemic data-volume memory issues or poor partition configurations causing an executor loss.

Question 23

How does Automatic Liquid Clustering determine which columns to use as clustering keys in a Unity Catalog-managed Delta table?

- A.** It automatically determines optimal clustering keys based on the type and order of column definition in the schema.
- B.** It leverages advanced sampling strategies to randomize column selection after uniformly balancing data across all files.
- C.** It intelligently selects clustering keys from predefined clustering columns specified during table creation.
- D.** It leverages Predictive Optimization to choose optimal clustering keys based on observed query behavior.

Correct answer: D

D. It leverages Predictive Optimization to choose optimal clustering keys based on observed query behavior.

Explanation

Automatic Liquid Clustering in Databricks is a feature designed to automatically optimize the physical layout of data in Delta tables based on the access patterns and metadata statistics. It leverages Predictive Optimization, which uses query behavior analytics to select clustering keys dynamically.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

Question 24

Which of the following offers a low-code, fully managed data ingestion solution from popular enterprise applications into Unity Catalog tables?

- A. Auto Loader
- B. Lakeflow Connect managed connector
- C. COPY INTO
- D. Databricks Connect

Correct answer: B

B. Lakeflow Connect managed connector

Explanation

Lakeflow Connect is Databricks' fully managed, low-code data ingestion solution designed specifically to discover, prepare, and replicate data from popular enterprise SaaS applications (like Salesforce, Workday, HubSpot, and ServiceNow) and relational databases directly into Unity Catalog tables. It operates on serverless compute, supports automated schema evolution, utilizes incremental processing / Change Data Capture (CDC), and integrates natively with Unity Catalog for end-to-end lineage and governance.

Why the other options are incorrect:

- **Auto Loader:** While it is an automated tool for incremental data ingestion, it is a code-based solution specifically designed for processing files (such as JSON, CSV, or Parquet) dropped into cloud object storage, rather than directly connecting to enterprise SaaS applications.
- **COPY INTO:** This is a declarative SQL command used to load data from cloud file locations into Delta tables in a batch manner. It requires writing SQL and does not connect natively to enterprise application APIs.
- **Databricks Connect:** This is a client library that allows developers to connect their favorite IDEs (like VS Code or PyCharm) and local development tools to Databricks clusters to run Spark code remotely. It is not an ingestion solution.

Question 25

A global airline stores flight booking events from multiple countries. Booking timestamps are recorded in local airport time zones.

To ensure accurate reporting and customer activity tracking, how should timestamps be standardized?

- A. Store timestamps without timezone context
- B. Keep timestamps in local time and apply timezone conversion only in BI tools
- C. Store all timestamps as `TIMESTAMP` type in UTC
- D. Store all timestamps as `STRING` type for readability

Correct answer: C

C. Store all timestamps as `TIMESTAMP` type in UTC

Explanation

In global data architectures, storing timestamps in **Coordinated Universal Time (UTC)** is the absolute best practice. Here is why this approach outperforms the other options:

- **A Single Source of Truth:** Storing data from multiple countries in a single, unshifting time standard removes all ambiguity. It allows you to seamlessly sequence events that happen across different continents (e.g., tracking if a passenger booked a connecting flight before or after their first leg landed).
- **Daylight Saving Time (DST) Immunity:** Local time zones change due to DST, which can cause duplicate hours in autumn or missing hours in spring. UTC does not observe DST, protecting your data from chronological anomalies.
- **Performance and Indexing:** Native `TIMESTAMP` types are stored internally as compact integers (typically microseconds or milliseconds since the Unix epoch). This makes sorting, filtering, and indexing incredibly fast compared to parsing `STRING` representations.

Why the other options are incorrect:

- **Storing as `STRING`:** While readable, it breaks date-time math. To filter data (e.g., `WHERE booking_date > '2026-01-01'`), the database engine must perform expensive string parsing, heavily degrading query performance.
- **Converting only in BI tools:** If you keep timestamps in local time without standardization at the database layer, downstream calculations (like global revenue per hour) become highly complex, error-prone, and slow, as the BI layer has to constantly map changing local offsets.

- **Storing without timezone context:** This results in data corruption. If an event is recorded as 10:00 AM without knowing if it was Tokyo, London, or New York time, the data becomes useless for global tracking.

Question 26

A data engineer has converted an external table to a managed table using the `ALTER TABLE ... SET MANAGED` command.

What are the key advantages of using `SET MANAGED` for this conversion?

Choose 2 answers

- A. Redirects path-based reads and writes to allow legacy code to function after conversion.
- B. Supports rolling back the converted managed table to an external table.
- C. Syncs both managed and external locations in parallel to support dual-write consistency after conversion.
- D. Continues using the original external location as the single source of truth for both reads and writes after conversion.
- E. Immediately deletes the original external storage location after conversion to save storage costs.

Correct answers: A, B

- A. Redirects path-based reads and writes to allow legacy code to function after conversion.
- B. Supports rolling back the converted managed table to an external table.

Explanation

The two correct key advantages are:

- **Rollback Failsafe:** If anything goes wrong or if there's a reason to revert, you can safely use the inverse command (`ALTER TABLE ... UNSET MANAGED`) to seamlessly roll back the managed table to an external table structure.
- **Path-Based Redirects:** To protect production workflows and prevent downtime, Databricks automatically redirects path-based reads and writes targeting the previous external storage location over to the new managed table location. This allows legacy code to continue functioning seamlessly without manual refactoring.

Why other options are incorrect:

- Continues using the original external location... / Syncs locations in parallel: A managed table inherently moves data management control entirely to Unity Catalog's managed storage root. It does not maintain a dual-write sync or treat the old location as a concurrent single source of truth.

- Deletes the original external storage location immediately: The original location is not immediately destroyed upon running the command. To prevent data loss, the data remains safely behind for 14 days.

Question 27

A data engineer is cleaning transaction data before loading it into a silver table. To improve data quality, the engineer uses the following PySpark command:

```
df.na.drop(how='all')
```

What is the effect of this operation?

- A. Removes rows only when all columns in the row are null
- B. Removes rows where all columns are non-null
- C. Replaces all null values with empty strings
- D. Removes rows where any column contains a null value

Correct answer: A

A. Removes rows only when all columns in the row are null

Explanation

By default, `df.na.drop()` (alias for `df.dropna()`) uses the argument `how='any'`, which drops a row if even a single column has a missing value. When you explicitly pass `how='all'`, you are altering that behavior to instruct Spark to only drop a row if **every single column** in that row evaluates to null (or NaN). If a row has even one valid, non-null value, it will be kept.

Why the other options are incorrect:

- **Removes rows where any column contains a null value** This describes the default behavior of `df.na.drop()` when no parameters are specified, or when configured as `df.na.drop(how='any')`.
- **Removes rows where all columns are non-null** This is incorrect because it describes the exact opposite of what the function is intended to do. This logic would eliminate perfectly complete rows of valid data while keeping rows full of missing values.
- **Replaces all null values with empty strings** This statement is incorrect because dropping completely removes rows from the dataset. If the data engineer wanted to swap out null values for empty strings, they would use `df.na.fill("")` or `df.na.replace()`.

Reference

<https://docs.databricks.com/aws/en/pyspark/reference/classes/dataframe/dropna>

Question 28

A data engineer has defined the following data quality constraint in a Delta Live Tables pipeline:

`CONSTRAINT valid_id EXPECT (id IS NOT NULL) _____`

Fill in the above blank so records violating this constraint will be added to the target table, and reported in metrics

A. ON VIOLATION ADD ROW

B. There is no need to add ON VIOLATION clause. By default, records violating the constraint will be kept, and reported as invalid in the event log

C. ON VIOLATION FAIL UPDATE

D. ON VIOLATION NONE

Correct answer: B

B. There is no need to add ON VIOLATION clause. By default, records violating the constraint will be kept, and reported as invalid in the event log

Explanation

By default, records that violate the expectation are added to the target dataset along with valid records, but violations will be reported in the event log

* Please note that Delta Live Tables (DLT) has been recently renamed to Lakeflow Declarative Pipeline, however, the current exam version may still refer to it as Delta Live Tables.

Study materials from our exam preparation course on Udemy:

- [Hands-on](#)

Question 29

A data engineering team needs to sync sales data from Snowflake into Databricks every 6 hours. Only records that have changed should be ingested to avoid unnecessary processing and duplication.

Which ingestion approach best meets this requirement?

- A. Export Snowflake data to files and append it to the target table every 6 hours
- B. Structured Streaming ingestion from Snowflake
- C. Incremental ingestion using Lakeflow managed connector for Snowflake
- D. Full refresh of the Snowflake table every 6 hours

Correct answer: C

C. Incremental ingestion using Lakeflow managed connector for Snowflake

Explanation

Databricks **Lakeflow Connect** provides fully managed, native connectors designed specifically to ingest data from SaaS applications and databases efficiently.

- **Incremental Ingestion & Efficiency:** On its first run, Lakeflow Connect performs a full load of the source table. For all subsequent runs, it leverages built-in incremental tracking (or Change Data Capture where supported) to identify and copy **only the records that have changed**. This directly prevents data duplication and unnecessary compute costs.

- **Scheduled Orchestration:** It functions natively with Databricks orchestration, allowing you to trigger ingestion pipelines effortlessly on a strict schedule (such as every 6 hours).

Why the other options are incorrect:

- **Full refresh of the Snowflake table every 6 hours:** This approach completely violates the requirement to avoid "unnecessary processing." Overwriting or replacing the entire dataset every 6 hours requires substantial, repetitive computational resources as the data grows.

- **Structured Streaming ingestion from Snowflake:** This requires writing and maintaining complex integration code compared to a fully managed, declarative connector.

- **Export Snowflake data to files and append it to the target table every 6 hours:** This represents a legacy, manual workaround. Exporting to a cloud staging layer (like AWS S3 or Azure Blob) using Snowflake Tasks/Streams and then reading it via Databricks introduces an extra layer of structural fragility, requires writing custom MERGE or APPEND logic, and creates unnecessary storage overhead.

Question 30

A data engineer is building a Lakeflow Spark Declarative Pipeline (formerly Delta Live Tables) to process product sales data. The pipeline needs to enforce the following data quality rules:

```
valid_products = {"valid_id": "products_id IS NOT NULL", "recent_sales": "date >= '2025-01-01',  
"quantity_within_range": "quantity BETWEEN 0 AND 1000"}
```

Any invalid records should still be written to the target, while metrics about these violations are captured by the pipeline.

Which of the following configurations would satisfy these requirements?

A. `@dp.table`

```
@dp.expect(valid_products)  
def silver_sales():  
    return spark.readStream("bronze_sales")
```

B. `@dp.table`

```
@dp.expect_or_fail(valid_products)  
def silver_sales():  
    return spark.readStream("bronze_sales")
```

C. `@dp.table`

```
@dp.expect_or_drop(valid_products)  
def silver_sales():  
    return spark.readStream("bronze_sales")
```

D. `@dp.table`

```
@dp.expect_all(valid_products)  
def silver_sales():  
    return spark.readStream("bronze_sales")
```

Correct answer: D

D. `@dp.table@dp.expect_all(valid_products)def silver_sales(): return spark.readStream("bronze_sales")`

Explanation

`dp.expect_all` enforces all the specified data quality rules, writes both valid and invalid records to the target table, and captures metrics about any rule violations.

`dp.expect` would not fully meet the requirements because it applies expectations individually but doesn't automatically enforce all of them together as a group. Similarly, `dp.expect_or_drop` removes individual invalid records, and `dp.expect_or_fail` stops the pipeline on individual rule violations. You can group multiple expectations together and specify collective actions using the functions `dp.expect_all_or_drop`, and `dp.expect_all_or_fail`.

Question 31

Which of the following locations hosts the Databricks workspace web application?

- A. Data plane
- B. Control plane
- C. Databricks-managed cluster
- D. Customer Cloud Account

Correct answer: B

B. Control plane

Explanation

According to the Databricks Lakehouse architecture, Databricks workspace is deployed in the control plane along with Databricks services like Databricks web application (UI), Cluster manager, workflow service, and notebooks.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

Question 32

What is the primary benefit of using spot instances in Databricks clusters?

- A. Significantly lower compute costs
- B. Increased security and compliance
- C. Guaranteed job execution time
- D. Reduced data storage latency

Correct answer: A

A. Significantly lower compute costs

Explanation

Spot instances are a cost-effective option provided by cloud providers (such as AWS, Azure, and GCP) that allow users to take advantage of unused compute capacity at a reduced price—often up to 90% less than on-demand instances. In Databricks, configuring clusters to use spot instances can substantially reduce operational expenses for workloads that are tolerant of potential interruptions.

Worker type ⓘ

Standard_D4ds_v5

16 GB Memory, 4 Cores



Min workers

2



Max workers

8



Spot instances ⓘ

However, while they offer significant cost savings, spot instances are not well-suited for time-sensitive or mission-critical workloads due to the risk of unexpected termination by the cloud provider.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

Question 33

A data engineering team manages a Delta Lake table in Unity Catalog called `employees`, with columns: `id`, `name`, `salary`, and `region`. They want to apply row filtering on this table so that only members of the HR team can access all records. If the table is queried by a non-HR team member, it should only show records in the France (FR) region. To achieve this, they implemented the following user-defined function:

```
CREATE FUNCTION fr_filter(region STRING)
RETURN IF(IS_ACCOUNT_GROUP_MEMBER('hr_team'), true, region='FR');
```

Which of the following commands can the team use to apply this function as a row filter to the table?

- A. `ALTER TABLE employees SET ROW FILTER fr_filter;`
- B. `SET ROW FILTER fr_filter ON TABLE employees TO COLUMN region`
- C. `ALTER TABLE employees SET ROW FILTER fr_filter ON (region);`
- D. `ALTER TABLE employees ALTER COLUMN region SET ROW FILTER fr_filter;`

Correct answer: C

C. `ALTER TABLE employees SET ROW FILTER fr_filter ON (region);`

Explanation

The correct command to apply the function as a row filter to the Delta Lake table is:

```
ALTER TABLE employees SET ROW FILTER fr_filter ON (region);
```

This command correctly associates the user-defined function `fr_filter` with the `region` column of the `employees` table, ensuring that access to rows is filtered based on the function's logic. In this setup, when a user queries the table, if they belong to the `hr_team` account group, they will be able to see all rows; otherwise, only rows where the `region` column equals 'FR' will be visible.

Question 34

A data engineer is building a Lakeflow Spark Declarative Pipeline (SDP) to ingest data from a PostgreSQL database, and want to use a streaming table for near-real-time analytics. The engineer assumes that streaming tables can directly enable continuous ingestion from JDBC sources.

Which statement is correct regarding this assumption?

- A. PostgreSQL can be consumed directly like Kafka in streaming tables
- B. Streaming tables support continuous JDBC ingestion from PostgreSQL
- C. JDBC sources are automatically treated as streaming sources in SDP
- D. PostgreSQL ingestion is only supported via batch JDBC, not native streaming

Correct answer: D

D. PostgreSQL ingestion is only supported via batch JDBC, not native streaming

Explanation

Standard JDBC connections are inherently static, batch-oriented data sources. Apache Spark's native JDBC connector does not support a continuous, incremental streaming mechanism. When you attempt to read from a PostgreSQL database using a traditional JDBC connection, Spark has no native way of tracking offsets, database transaction logs, or row-level deltas in a continuous stream.

Therefore, if a data engineer wants to bring data from PostgreSQL directly into an SDP using a traditional Spark JDBC configuration, it must be performed as a **batch/snapshot read** (such as utilizing the `spark.read.format("jdbc")` function).

To achieve true, near-real-time streaming from PostgreSQL into Databricks Lakeflow, you cannot use raw JDBC. Instead, you must use **Lakeflow Connect**, which relies on native database logical replication (WAL/Change Data Capture) rather than standard JDBC polling.

Why the Other Options are Incorrect

- **"Streaming tables support continuous JDBC ingestion from PostgreSQL"**

- Streaming tables are designed to incrementally append or update data coming from streaming sources (like Cloud Object Storage via Auto Loader/read_files or message queues like Kafka). They cannot natively force a raw JDBC database connection into a continuous stream because the JDBC source itself lacks streaming semantics.

- **"JDBC sources are automatically treated as streaming sources in SDP"**

- In SDP, Spark cannot automatically convert a static table from a relational database into a streaming source. If you attempt to wrap a standard JDBC dataset in a STREAM format, the pipeline will fail with an

error stating that the source does not support streaming.

- "PostgreSQL can be consumed directly like Kafka in streaming tables"

- Kafka is built from the ground up as a distributed append-only log with inherent message offsetting, making it natively compatible with Spark's Structured Streaming engine. PostgreSQL is a relational database; it does not naturally expose a pub/sub message log via standard JDBC. Treating it like Kafka would require an intermediary CDC tool or a fully configured ingestion engine like Lakeflow Connect.

Question 35

A data engineering team is building a reusable product filtering query in Databricks SQL. The query must accept a runtime input value for `product_id` using a named parameter called “pid” so that different values can be passed without changing the SQL logic. Which query correctly defines and uses the named parameter?

A.

```
SELECT *  
FROM products  
WHERE product_id = $pid;
```

B.

```
SELECT *  
FROM products  
WHERE product_id = :pid;
```

C.

```
SELECT *  
FROM products  
WHERE product_id = pid;
```

D.

```
SELECT *  
FROM products  
WHERE product_id = ${var.pid};
```

Correct answer: B

Explanation

In **Databricks SQL**, named parameter markers are defined using a leading colon (:) followed by the parameter name (e.g., :parameter_name). The mandatory preceding colon explicitly tells the parser to differentiate the parameter namespace from column names and other SQL variables.

Therefore, to use the parameter named pid, the correct syntax is :pid.

Why the other options are incorrect:

- **B (product_id = pid):** Without a prefix, Databricks SQL evaluates pid as a column identifier rather than a dynamic query parameter.
- **C (product_id = \$pid):** The \$ prefix is invalid for Databricks SQL named parameter markers.
- **D (product_id = \${var.pid}):** This syntax mimics variable substitution in Declarative Automation Bundles (DABs) rather than native SQL parameterization.

Question 36

A data engineering team at a supply chain company uses Lakeflow Declarative Pipelines (formerly Delta Live Tables) to manage inventory data. The team maintains an append-only streaming table, `inventory_raw`, that stores raw inventory status information, with columns: `product_id`, `quantity`, and `event_timestamp`.

A data engineer is tasked with creating a new table, `inventory_latest`, to capture near real-time changes in product inventory levels from `inventory_raw`. The new table will include the columns: `product_id`, `current_quantity`, and `updated_timestamp`.

Which of the following types of objects would be most suitable to implement the `inventory_latest` table?

- A. Streaming table
- B. Materialized view
- C. Temporary view
- D. Streaming view

Correct answer: A

A. Streaming table

Explanation

The most suitable object to implement the `inventory_latest` table is a Streaming table, because it is designed to continuously capture and update near real-time changes from an append-only source like `inventory_raw`. Implementing `inventory_latest` as a Streaming table allows you to merge incoming changes or apply a CDC (Change Data Capture) feed from `inventory_raw` so that the table always reflects the latest inventory state per `product_id`. Each new event-whether an update or insertion-can be applied in real time, updating `current_quantity` and `updated_timestamp` without rebuilding the entire table, which is the main advantage over a materialized view or temporary view.

Materialized Views (formerly known as Live Tables) provide batch-oriented or scheduled incremental processing for precomputed queries rather than continuously updating individual records in real time. Temporary views, in contrast, are ephemeral and not suited for persistent state tracking.

Question 37

Given the following Structured Streaming query:

```
(spark.readStream
  .table("cleanedOrders")
  .groupBy("productCategory")
  .agg(sum("totalWithTax"))
  .writeStream
  .option("checkpointLocation", checkpointPath)
  .outputMode("complete")
  .table("aggregatedOrders")
)
```

Which of the following best describe the purpose of this query in a Medallion Architecture?

- A. The query is performing a hop from a Bronze table to a Silver table
- B. The query is performing raw data ingestion into a Bronze table
- C. The query is performing data transfer from a Gold table into a production application
- D. The query is performing a hop from Silver layer to a Gold table

Correct answer: D

- D. The query is performing a hop from Silver layer to a Gold table

Explanation

The above Structured Streaming query creates business-level aggregates from clean orders data in the silver table `cleanedOrders`, and loads them in the gold table `aggregatedOrders`.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

- [Hands-on](#)

Question 38

A finance data analyst requires read-only access to all existing and future tables within the Unity Catalog schema `enterprise.reporting`. Access must be limited exclusively to this schema, with no permissions granted to other schemas in the enterprise catalog.

Which SQL commands will correctly grant this access?

A.

```
GRANT BROWSE ON SCHEMA enterprise.reporting TO finance-analyst;  
GRANT USE SCHEMA ON SCHEMA enterprise.reporting TO finance-analyst;  
GRANT USE CATALOG ON CATALOG enterprise TO finance-analyst;
```

B.

```
GRANT SELECT ON SCHEMA enterprise.reporting TO finance-analyst;  
GRANT USE SCHEMA ON SCHEMA enterprise.reporting TO finance-analyst;  
GRANT USE CATALOG ON CATALOG enterprise TO finance-analyst;
```

C.

```
GRANT SELECT ON CATALOG enterprise TO finance-analyst;  
GRANT USE SCHEMA ON CATALOG enterprise TO finance-analyst;  
GRANT USE CATALOG ON CATALOG enterprise TO finance-analyst;
```

D.

```
GRANT USE SCHEMA ON SCHEMA enterprise.reporting TO finance-analyst;  
GRANT USE CATALOG ON CATALOG enterprise TO finance-analyst;
```

Correct answer: B

Explanation

In **Databricks Unity Catalog**, privileges are structured hierarchically (catalog > schema > table/view). To access any data object, a user must have the appropriate container permissions at every level above it:

- **USE CATALOG**: Required on the parent catalog (enterprise) to traverse down to its contained schemas. It does not grant read or write access to any other data objects inside the catalog.
- **USE SCHEMA**: Required on the specific schema (enterprise.reporting) to look up objects inside that schema.
- **SELECT**: Granting SELECT at the **schema level** automatically propagates read-only access to all **existing and future tables and views** within that specific schema. This completely satisfies the requirement without explicitly granting permissions table-by-table.

Question 39

A data engineer is tasked with calculating both the total and average salary of employees, grouped by their department. They use PySpark's `groupBy` and wish to apply multiple aggregations to each group.

Complete the following code snippet:

```
result_df = df.groupBy("department")._____(  
    sum("salary").alias("total_salary"),  
    avg("salary").alias("average_salary")  
)
```

What function should be used to complete the code?

A. `withColumn`

B. `select`

C. `agg`

D. `filter`

Correct answer: C

C. `agg`

Explanation

The `agg()` function is used after a `groupBy()` operation when applying one or more aggregate functions to grouped data. In this scenario, the data engineer is calculating both the sum and average of salaries, so `agg()` is the correct function.

Question 40

A data engineer uses the Catalog Explorer in the Databricks workspace to manage object privileges in Unity Catalog for a retail company, as follows:

- On catalog **sales**, they open Permissions and grant USE CATALOG to **retail-analysts**
- On schema **transactions**, they open Permissions and grant USE SCHEMA to **retail-analysts**
- On the table **daily_revenue**, they open Permissions and grant SELECT to **retail-analysts**

Which of the following correctly describe the result of these actions?

- A.** The retail-analysts group will be able to query all tables in the transactions schema because USE SCHEMA automatically grants SELECT on every table in the schema.
- B.** The retail-analysts group will be able to read data from daily_revenue using SQL warehouses, but will not be able to create, alter, or delete objects in the sales catalog or transactions schema.
- C.** The retail-analysts group will be able to create new tables in the transactions schema because they were granted both USE CATALOG and USE SCHEMA.
- D.** The permissions configured through the Catalog Explorer UI will not take effect until the data engineer executes the generated GRANT SQL commands manually in the Databricks SQL editor.

Correct answer: B

- B.** The retail-analysts group will be able to read data from daily_revenue using SQL warehouses, but will not be able to create, alter, or delete objects in the sales catalog or transactions schema.

Explanation

In Databricks Unity Catalog, securing and accessing data follows an **explicit, hierarchical privilege model**. Here is why this statement is correct based on the actions taken:

- **Read Access Granted Clearly:** To read a table, a user or group needs the USE CATALOG privilege on its parent catalog, USE SCHEMA on its parent schema, and SELECT on the table itself. The data engineer explicitly granted all three of these matching prerequisites (sales catalog -> transactions schema -> daily_revenue table).

- **No Write/Modify Privileges:** The USE CATALOG and USE SCHEMA privileges only allow a user to traverse or look inside those containers; they do not grant any permission to write, modify, or delete. To create tables, the group would explicitly need the CREATE TABLE privilege on the schema. To alter or delete, they would need ownership or explicit modification rights (like MODIFY or ALL PRIVILEGES).

Question 41

A data engineer uses Declarative Automation Bundles (formerly Databricks Asset Bundles) to manage workflows across dev and prod environments.

The CI/CD process must verify bundle configurations on dev for every pull request. On merges to the prod branch, the system should automatically deploy to production.

Which set of actions meets these requirements?

- A.** In PR pipelines, execute `databricks bundle run validate -t dev`;
On prod, execute `databricks bundle run deploy -t prod`
- B.** In PR pipelines, execute `databricks bundle deploy -t dev -verify`;
On prod, execute `databricks bundle deploy -t prod`
- C.** In PR pipelines, execute `databricks bundle validate -t dev`;
On prod, execute `databricks bundle deploy -t prod`
- D.** In PR pipelines, execute `databricks bundle deploy -t dev`;
On prod, execute `databricks bundle run validate -t prod`

Correct answer: C

C. In PR pipelines, execute `databricks bundle validate -t dev`; On prod, execute `databricks bundle deploy -t prod`

Explanation

- `databricks bundle validate -t`: This command is used to syntactically and structurally verify that the bundle's YAML configuration (`databricks.yml`) is correct and valid for the specified environment target without actually deploying any resources to the Databricks workspace. It is the recommended best practice for Continuous Integration (CI) pull request gatekeepers.

- `databricks bundle deploy -t`: This command handles the actual generation, compilation, and deployment of the bundle assets (such as jobs, pipelines, and source files) into the designated target workspace environment. This fits the requirement for an automated production deployment upon a branch merge.

-

Why the others are incorrect:

- Commands structured as `databricks bundle run validate` or `databricks bundle run deploy` are invalid syntax; `databricks bundle run` is explicitly used to execute an already deployed resource (like a specific

workflow job or pipeline).

- The --verify flag is not a valid parameter for the databricks bundle deploy command.

Question 42

A data engineer needs to enforce that age is greater than 18 on an existing Delta table named users; Which SQL statement correctly adds this constraint?

- A. ALTER TABLE users ALTER COLUMN age SET (age > 18);
- B. ALTER TABLE users CHECK (age > 18)
- C. ADD CONSTRAINT eligible_adult ON TABLE users CHECK (age > 18)
- D. ALTER TABLE users ADD CONSTRAINT eligible_adult CHECK (age > 18)

Correct answer: D

D. ALTER TABLE users ADD CONSTRAINT eligible_adult CHECK (age > 18)

Explanation

In Databricks and Delta Lake, you manage constraints using standard SQL ALTER TABLE syntax. Delta Lake supports two types of constraints: NOT NULL and CHECK constraints.

To add a CHECK constraint to an existing table, the required syntax specifies the table name, the ADD CONSTRAINT clause with a unique name for the constraint, and the CHECK condition itself.

```
ALTER TABLE table_name ADD CONSTRAINT constraint_name CHECK (condition);
```

Before enforcing the constraint, Delta Lake will automatically verify that all existing rows in the table meet the condition. If any existing row has an age less than or equal to 18, the command will fail.

The other options are incorrect because they represent invalid syntax formats.

Question 43

Which of the following functionalities can be performed in Git folders?

- A. Create new remote Git repositories
- B. Delete branches
- C. Create pull requests
- D. Pull changes from a remote Git repository

Correct answer: D

- D. Pull changes from a remote Git repository

Explanation

Git folders supports git Pull operation. It is used to fetch and download content from a remote repository and immediately update the local repo to match that content.

Study materials from our exam preparation course on Udemy:

- [Hands-on](#)

Question 44

An IoT platform stores device telemetry in a bronze table. The column payload is a STRING containing JSON data like:

```
{"device_id": "A1", "temperature": 22.5, "humidity": 60}
```

A data engineer needs to extract fields for a silver table so they can be queried efficiently.

Which TWO queries correctly extract data from the JSON?

A.

```
SELECT get_json_object(payload, '$.device_id') AS device_id,  
       get_json_object(payload, '$.temperature') AS temperature  
FROM bronze_iot;
```

B.

```
SELECT json_extract(payload, 'device_id') AS device_id,  
       json_extract(payload, 'temperature') AS temperature  
FROM bronze_iot;
```

C.

```
SELECT payload:device_id AS device_id,  
       payload:temperature AS temperature  
FROM bronze_iot;
```

D.

```
SELECT payload.device_id AS device_id,  
       payload.temperature AS temperature  
FROM bronze_iot;
```

E.

```
SELECT from_json(payload,  
                 'device_id STRING, temperature STRING, humidity STRING') AS  
       parsed_payload  
FROM bronze_iot;
```

Correct answers: A, C

Explanation

The **TWO correct queries** that extract data from the JSON string in Databricks SQL are:

1. payload:device_id (The Colon Sign Operator)

Databricks SQL features native semi-structured data extraction using the `:` syntax. When you use `column:field`, it acts as a shortcut to parse a STRING containing JSON (or a VARIANT type) and extract the specified key directly. It is highly readable and standard practice in modern Databricks pipelines.

2. get_json_object(payload, '\$.field')

This is a standard built-in function designed specifically to parse JSON strings using JSONPath syntax. The `$` represents the root JSON object, followed by `.field` to point to the desired key. It extracts the value efficiently and returns it as a string.

Why the other options are incorrect:

- `payload.device_id` (Dot Notation)

Dot notation only works out-of-the-box if the column is already a native complex type like a **STRUCT** or a **VARIANT**. Because the prompt states that the payload column is a **STRING**, regular dot notation will fail with a compilation error because strings do not have native sub-attributes.

- `json_extract(payload, 'device_id')`

`json_extract` is a valid function in Presto/Trino and Presto-flavored SQL dialects, but it is **not** a valid Databricks SQL built-in function. Running this in Databricks throws an "unresolved attribute / function not found" error.

- `from_json(payload, 'device_id STRING...')`

While `from_json` is a completely valid Databricks function, this query contains two terminal errors:

- **Syntax Error:** The schema definition string is broken. It opens with a single quote ('device_id...') but is missing the closing single quote before the closing parenthesis.

- **Logical Error:** `from_json` parses the string into a single nested structural object (a **STRUCT**). To actually extract the individual fields out as columns for the silver table, you would still need to append dot notation to the outside of the function (e.g., `from_json(...).device_id`). As written, it does not output separate `device_id` and `temperature` columns.

Question 45

A data engineer converts an external Delta table into a managed table in a production environment using the `ALTER TABLE ... SET MANAGED` command. After the conversion is completed, the streaming query stops receiving new data with the following error:

“DELTA_STREAMING_INTERRUPTED_BY_MANAGED_TABLE_CONVERSION: The table at <path> has been converted to a Unity Catalog managed table. The stream has been stopped to ensure data consistency.”

How should the data engineer fix this issue?

- A. Roll back the converted managed table to an external table as this feature is supported by the `SET MANAGED` command.
- B. Run the `VACUUM` command on the converted table to remove stale files and restore streaming state.
- C. Run the `REPAIR TABLE` command to update the checkpointing offset of the converted table.
- D. Restart the stream and it will automatically resume from the last committed offset using the converted table.

Correct answer: D

D. Restart the stream and it will automatically resume from the last committed offset using the converted table.

Explanation

When you use the `ALTER TABLE ... SET MANAGED` command in Databricks, the underlying data files are moved from your external storage location to the managed storage location defined in Unity Catalog.

To prevent data corruption or missing updates during this migration, Databricks deliberately interrupts active streams reading from or writing to the table. The error message

DELTA_STREAMING_INTERRUPTED_BY_MANAGED_TABLE_CONVERSION is built-in safety feedback telling you that the structure has changed. To fix it, you simply need to **restart the streaming job**. Once restarted with the same configuration, it will leverage Unity Catalog's automatic path-based redirection and pick up right where it left off using the newly managed table.

Why the other options are incorrect:

- **"Run the `REPAIR TABLE` command to update the checkpointing offset..."** `REPAIR TABLE` is designed to scan file storage for new partition directories that were added outside of the catalog and register them to the metastore. It does not touch, update, or fix structured streaming checkpoints, which are managed independently by the streaming engine via cloud storage paths.

- **"Run the `VACUUM` command on the converted table to remove stale files..."** `VACUUM` is used to permanently delete data files that have been logically removed from a Delta table via deletes or

overwrites and are older than a specific retention threshold. Running it will not reset a streaming state or clear a streaming block.

- **"Roll back the converted managed table to an external table as this feature is supported by the SET MANAGED command."** Even though UNSET MANAGED makes a rollback technically possible, doing so does not solve the data engineer's goal of moving to a managed table in production. Rolling back would undo the migration they just performed, rather than fixing the stream to work with the newly converted table.

Question 46

Which of the following will utilize Gold tables as their source?

A. Dashboards

B. Silver tables

C. Auto loader

D. Bronze tables

Correct answer: A

A. Dashboards

Explanation

Gold tables provide business level aggregates often used for reporting and dashboarding, or even for Machine learning.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

Question 47

The engineering team is managing a large Delta Lake table with frequent updates and deletes. They notice that query performance is degrading over time due to an increase in small data files. To address this, they decide to run the OPTIMIZE command with Z-Order indexing on columns frequently used in filters.

What type of resource optimization should the engineering team prioritize to ensure these commands execute efficiently?

- A. Storage Optimized
- B. Compute Optimized
- C. Memory Optimized
- D. GPU Optimized

Correct answer: B

B. Compute Optimized

Explanation

The OPTIMIZE command compacts small files into larger ones for better access patterns. Z-Order indexing further sorts data based on specific columns to improve query pruning. Both operations require substantial computational resources for scanning and writing data. Therefore, compute-optimized resources provide the necessary CPU power and parallelism to efficiently process these tasks. While storage and memory are important, the main bottleneck during optimization is CPU-intensive compute operations.

Worker type ⓘ

Standard_D4ds_v5

16 GB Memory, 4 Cores

▼

General purpose

Standard_DS3_v2	14 GB Memory, 4 Cores
Standard_DS4_v2	28 GB Memory, 8 Cores
Standard_DS5_v2	56 GB Memory, 16 Cores

Memory optimized

Standard_DS12_v2	28 GB Memory, 4 Cores
Standard_DS13_v2	56 GB Memory, 8 Cores

Storage optimized

Standard_L8s_v3	64 GB Memory, 8 Cores
Standard_L16s_v3	128 GB Memory, 16 Cores
Standard_L32s_v3	256 GB Memory, 32 Cores

Compute optimized

► Advanced options

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

Question 48

A data engineer is working with a PySpark DataFrame called `productsDf` that contains multiple columns, including one named `price`. They want to return all columns from the DataFrame while displaying the price column with a temporary alias `unit_price` in the query output, without modifying the original DataFrame schema.

Which PySpark code fragment meets this requirement?

- A. `productsDf.select(col("*"), col("price").alias("unit_price"))`
- B. `productsDf.selectExpr("price as unit_price")`
- C. `productsDf = productsDf.withColumnRenamed("price", "unit_price")`
- D. `productsDf.select(col("price").alias("unit_price"))`

Correct answer: A

A. `productsDf.select(col("*"), col("price").alias("unit_price"))`

Explanation

Here is why this specific fragment works while the others fail to meet the exact requirements:

- **`productsDf.select(col("*"), col("price").alias("unit_price"))`**:

- `col("*")` selects **all original columns** from the DataFrame.

- `col("price").alias("unit_price")` creates a temporary, duplicated column containing the price data but renames it to `unit_price` in the output.

- It preserves the rest of the columns, leaves the original DataFrame schema completely unmodified, and outputs the result immediately.

Why the other options are incorrect:

- **`productsDf.selectExpr("price as unit_price")`**: This will only return a DataFrame with a single column (`unit_price`), dropping all the other columns.

- **`productsDf.select(col("price").alias("unit_price"))`**: Like `selectExpr`, this isolates the price column and excludes all other columns from the output.

- **`productsDf = productsDf.withColumnRenamed("price", "unit_price")`**: This actually modifies the schema by replacing the price column name with `unit_price`, meaning the original price column name is

lost in the resulting DataFrame. Furthermore, it assigns it back to `productsDf`, overwriting the variable.

Question 49

A data architect is designing a hybrid data platform that must securely connect to on-premises databases. These databases need to be queried using SQL from within Databricks, and the platform must remain within the enterprise's custom-defined network for compliance reasons.

Which compute type should the architect choose to support this architecture?

- A. Serverless compute for notebooks
- B. Serverless compute for jobs
- C. Serverless SQL Warehouse
- D. Pro SQL Warehouse

Correct answer: D

D. Pro SQL Warehouse

Explanation

Pro SQL Warehouses support secure connectivity to custom-defined networks, including hybrid architectures where data resides both in the cloud and on-premises. This setup is ideal for scenarios requiring private network access to databases or event-driven architectures.

Serverless options are managed by Databricks and typically do not operate within custom VPCs, making them unsuitable for such scenarios.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

Question 50

A data engineer needs to use Auto Loader to ingest JSON files that may contain new fields over time.

How should Auto Loader be configured to allow the table schema to evolve as new columns are detected during processing?

- A. Auto Loader cannot detect the addition of new columns in input files during processing.
- B. Set `mergeSchema` to `true` to enable Delta Lake merge schemas during writes
- C. Configure `cloudFiles.schemaLocation`, and set `cloudFiles.schemaEvolutionMode=addNewColumns` to support automatic schema evolution
- D. Set `inferColumnTypes` to `true` to automatically detect new columns with their data types.

Correct answer: C

C. Configure `cloudFiles.schemaLocation`, and set `cloudFiles.schemaEvolutionMode=addNewColumns` to support automatic schema evolution

Explanation

Databricks Auto Loader uses a set of specific configuration properties prefixed with `cloudFiles` to manage schema inference and evolution when ingesting data (such as JSON or CSV files).

To enable automatic schema evolution, Auto Loader requires two core setups:

- **`cloudFiles.schemaLocation`**: A directory path where Auto Loader stores the inferred schema and tracks changes over time. This is mandatory for schema evolution to work.

- **`cloudFiles.schemaEvolutionMode`**: This controls how Auto Loader handles new columns. Setting it to **`addNewColumns`** instructs the engine to automatically append any new fields discovered in the source files to the existing table schema without failing the streaming job.

Code Example

In practice, a data engineer would configure the streaming DataFrame reader like this:

```
# Streaming ingestion with Auto Loader schema evolution
df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/path/to/schema/checkpoint")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .load("/path/to/source/json/files"))

# Writing to a Delta table
query = (df.writeStream
```

```
.option("checkpointLocation", "/path/to/stream/checkpoint")  
.table("my_evolution_table"))
```

Why the other options are incorrect

- **"Auto Loader cannot detect..."**: This is false. Schema inference and evolution are core functionalities of Auto Loader specifically designed to handle data drift in file formats like JSON and CSV.
- **mergeSchema = true**: While mergeSchema is a valid Delta Lake option used during writes to merge schemas, Auto Loader handles schema evolution on the read/ingestion side using cloudFiles.schemaEvolutionMode. Relying on Delta's mergeSchema alone without configuring Auto Loader's schema evolution mode will result in the streaming read failing or ignoring the new columns before they ever reach the write stage.
- **inferColumnTypes = true**: While Auto Loader can infer column data types, this is not a valid configuration option for driving automatic schema evolution.

Question 51

A data engineer is analyzing a Spark job via the Spark UI. They have the following summary metrics for 27 completed tasks in a particular stage

Summary Metrics for 27 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	10 s	20 s	25 s	30 s	45 s
GC Time	20.0 ms	32.0 ms	44.0 ms	51.0 ms	1.9 s
Input Size / Records	78 KiB / 16	179 KiB / 49	183.9 KiB / 55	186.7 KiB / 67	203 KiB / 76
Shuffle Write Size / Records	1.4 KiB / 16	6.1 KiB / 49	6.9 KiB / 55	8.1 KiB / 67	12.8 KiB / 76

Which conclusion can the data engineer draw from the above statistics?

- A. The cluster does not have enough resources to handle the workload.
- B. The system is spending too much time on garbage collection (GC)
- C. The data is not evenly distributed across tasks, indicating data skew.
- D. The stage is running normally without any major issues.

Correct answer: D

D. The stage is running normally without any major issues.

Explanation

To determine if a Spark stage has issues like data skew, resource starvation, or excessive garbage collection (GC), we look at the distribution of metrics across the percentiles (Min, 25th, Median, 75th, Max).

- **No Data Skew:** The Input Size / Records and Shuffle Write Size / Records are beautifully and evenly distributed. For instance, the median records processed per task is **55**, and the maximum is **76**. If there were severe data skew, you would see a massive gap where the Max value is orders of magnitude higher than the Median (e.g., Median = 55, Max = 1,000,000).

- **Healthy Garbage Collection (GC) Time:** The tasks spend a maximum of **59.0 ms** on garbage collection out of a **45 s** maximum task duration. As a rule of thumb in Spark performance tuning, GC time is only considered a major issue if it exceeds **10%** of the total task duration. Here, it represents less than **0.2%** of the execution time.

- **Even Task Durations:** Task durations are tightly clustered between **10 s (Min)** and **45 s (Max)**, with a median of **25 s**. This indicates that the workload is well-balanced across the cluster workers.

Question 52

A production environment has an S3 bucket receiving thousands of image files daily in different formats (.png, .jpg, .gif). A data engineer has been tasked with modifying the following streaming ingestion script to ensure only **.jpg** files are processed.

```
df = spark.readStream
    .format("cloudFiles") \
    .option("cloudFiles.format", "binaryFile") \
    .option("_____", "*.jpg") \
    .load("s3://shop/raw/invoices/")
```

Which option correctly fills in the blank to meet the specified requirement ?

- A. cloudFiles.fileExtension
- B. cloudFiles.pathGlobFilter
- C. fileExtension
- D. pathGlobFilter

Correct answer: D

D. pathGlobFilter

Explanation

The pathGlobFilter option allows you to filter input files based on a glob pattern, such as ***.jpg**, when using Auto Loader.

Study materials from our exam preparation course on Udemy:

- [Lecture](#)

Question 53

A data engineering team maintains a reporting job that should automatically refresh whenever a Delta table is updated by upstream ingestion pipelines. The goal is to react to changes in the table so that downstream reports remain consistent and up to date as new data arrives.

What trigger configuration should the team use for this job?

- A. File arrival trigger
- B. Scheduled trigger
- C. Continuous trigger
- D. Table update trigger

Correct answer: D

- D. Table update trigger

Explanation

In Lakeflow Jobs, the **Table update trigger** is specifically built to monitor source tables and automatically trigger downstream jobs as soon as new data changes-such as updates, merges, or deletes-occur.

This behavior allows the team to eliminate scheduled guesswork or continuous cluster costs.

Why the other options are incorrect:

- **File arrival trigger:** This triggers a job when raw files are uploaded to a cloud storage path or Unity Catalog volume, rather than responding to internal database updates within a Delta table.
- **Continuous trigger:** This keeps a job permanently running by instantly launching a new run as soon as the previous one finishes or fails, which is inefficient for non-streaming, event-driven report refreshes.
- **Scheduled trigger:** This relies on a fixed time interval (such as a cron schedule) and does not react dynamically to actual data updates.

Question 54

Which of the following describes Cron syntax in Databricks Jobs?

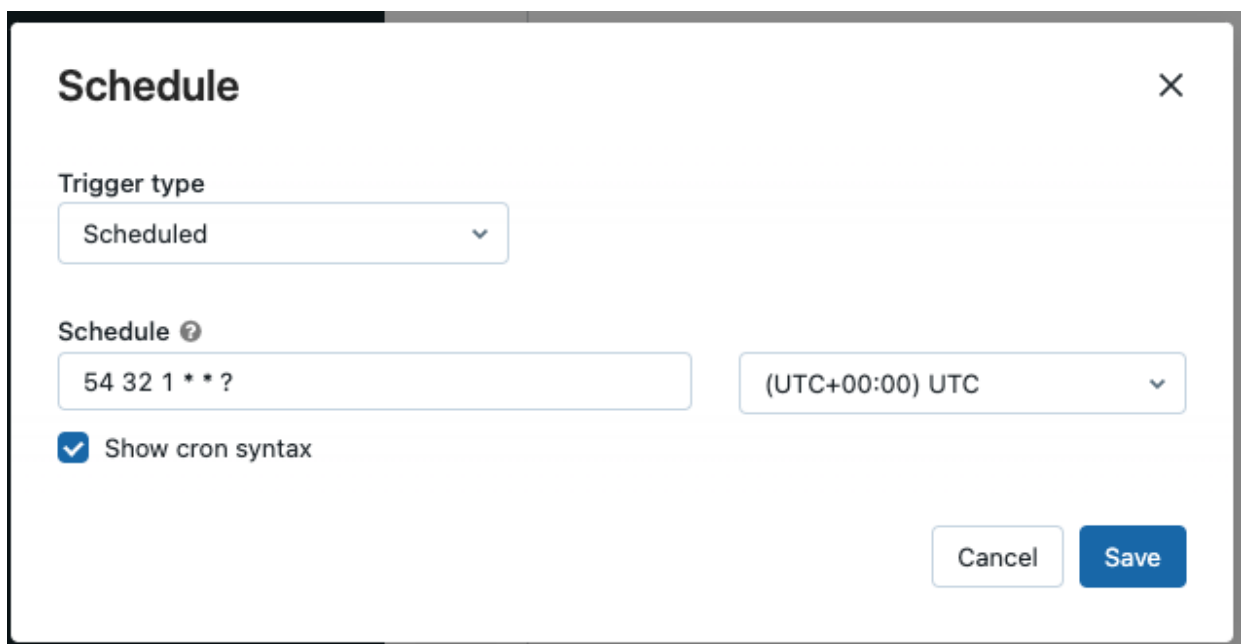
- A. It's an expression to represent the maximum concurrent runs of a job
- B. It's an expression to represent the retry policy of a job
- C. It's an expression to represent complex job schedule that can be defined programmatically
- D. It's an expression to represent the run timeout of a job

Correct answer: C

C. It's an expression to represent complex job schedule that can be defined programmatically

Explanation

To define a schedule for a Databricks job*, you can either interactively specify the period and starting time, or write a Cron Syntax expression. The Cron Syntax allows to represent complex job schedule that can be defined programmatically



The screenshot shows a 'Schedule' dialog box with the following fields and controls:

- Trigger type:** A dropdown menu with 'Scheduled' selected.
- Schedule:** A text input field containing the cron expression '54 32 1 * * ?'.
- Time zone:** A dropdown menu with '(UTC+00:00) UTC' selected.
- Show cron syntax:** A checked checkbox.
- Buttons:** 'Cancel' and 'Save' buttons at the bottom right.

* Please note that Databricks Jobs has been recently renamed to Lakeflow Jobs, however, the current exam version may still refer to it as Databricks Jobs.

Study materials from our exam preparation course on Udemy:

[Hands-on](#)

Question 55

A data engineer has been tasked with consuming a stream of events from a Kafka topic named `events_topic`, which is hosted on a remote Kafka broker at `host:port`.

Which of the following code snippets correctly establishes a streaming DataFrame in PySpark to read from this Kafka topic?

A.

```
eventsStream = (spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "host:port")
    .option("subscribe", "events_topic")
    .option("startingOffsets", "latest")
    .load()
)
```

B.

```
eventsStream = spark.readStream.format("kafka").load("host:port", "events_topic",
"latest")
```

C.

```
eventsStream = (spark.readStream
    .format("cloud_files")
    .option("cloudFiles.format", "kafka")
    .option("cloudFiles.bootstrap.servers", "host:port")
    .option("cloudFiles.subscribe", "events_topic")
    .option("cloudFiles.startingOffsets", "latest")
    .load()
)
```

D.

```
eventsStream = spark.readStream.kafka("host:port", "events_topic", "latest")
```

Correct answer: A

Explanation

The correct approach uses PySpark's `readStream` with format "kafka", and `.option()` methods to specify the necessary configurations, such as:

- `kafka.bootstrap.servers` to identify the Kafka cluster
- `subscribe` to define the target Kafka topic, and
- `startingOffsets` to control the consumption starting point. This can be set to "latest" or "earliest" to determine where to begin reading.